

LAB7

```
#include <stdio.h>
```

```
struct Queue
{
    char data;
    struct Queue *next;
};
```

```
struct Queue *Front = NULL;
struct Queue *Rear = NULL;
```

```
struct Queue* enqueue(char x);
struct Queue* dequeue();
int peek();
int isEmpty();
void display();
```

```
int main()
{
    Front = enqueue(10);
    Front = enqueue(100);
    Front = enqueue(110);
    Front = enqueue(102);

    Front = dequeue();

    display();

    int x = peek();
    printf("%d\n", x);

    if (isEmpty())
    {
        printf("The stack is empty.\n");
    }
    else
    {
        printf("Stack is not empty.\n");
    }
}
```

```
}  
}
```

```
struct Queue* enqueue(char x)  
{  
    struct Queue *t = (struct Queue*) malloc (sizeof(struct Queue));  
    t->data = x;  
  
    if (Front==NULL)  
    {  
        Front = t;  
        t->next = NULL;  
        Rear = t;  
        return Front;  
    }  
  
    Rear->next = t;  
    t->next = NULL;  
    Rear = Rear->next;  
  
    return Front;  
}
```

```
struct Queue* dequeue()  
{  
    if (Front == NULL)  
    {  
        printf("Your stack is empty!\n");  
        return NULL;  
    }  
  
    struct Queue *p = Front;  
    Front = Front->next;  
    free(p);  
  
    return Front;  
}
```

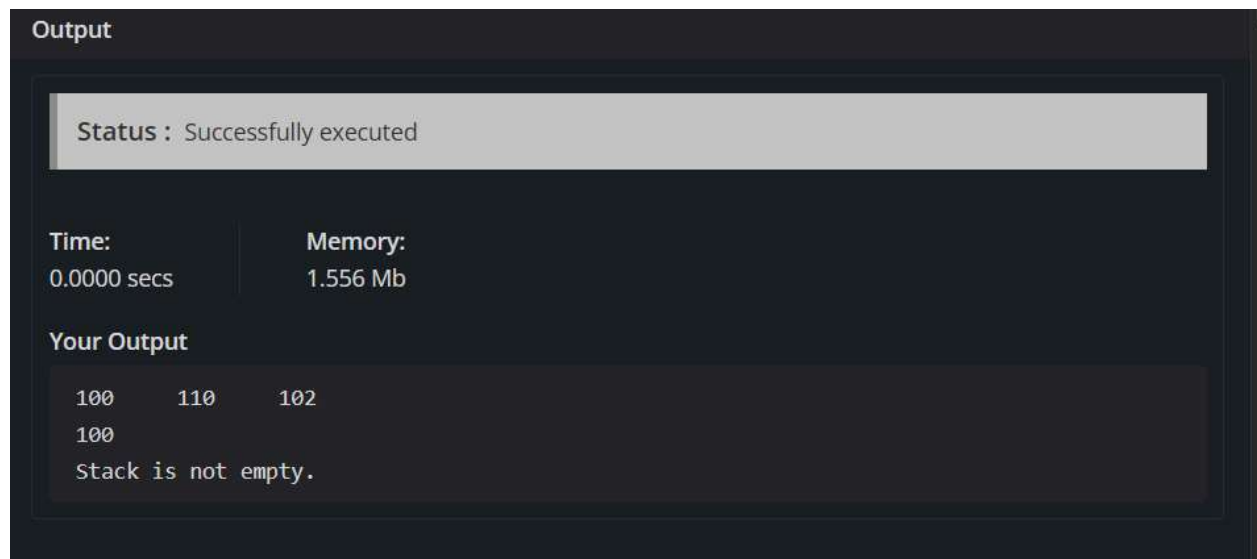
```
int peek()
```

```
{
    if (Front == NULL)
    {
        printf("Your stack is empty!\n");
        return -1;
    }
    return Front->data;
}
```

```
int isEmpty()
{
    if (Front == NULL && Rear == NULL)
    {
        return 1;
    }
    else
    {
        return 0;
    }
}
```

```
void display()
{
    struct Queue *po = Front;

    while (po != NULL)
    {
        printf("%d\t", po->data);
        po = po->next;
    }
    printf("\n");
}
```



```
#include <stdio.h>
```

```
struct Queue
```

```
{  
    int data;  
    int priority;  
    struct Queue *next;  
};
```

```
struct Queue *Front = NULL;
```

```
struct Queue *Rear = NULL;
```

```
struct Queue* enqueue(int x, int priority);
```

```
struct Queue* dequeue();
```

```
int peek();
```

```
void display();
```

```
int main()
```

```
{  
    Front = enqueue(10, 0);  
    Front = enqueue(100, 3);
```

```
Front = enqueue(110, 2);  
Front = enqueue(102, 1);
```

```
//Front = dequeue();
```

```
display();
```

```
int x = peek();  
printf("%d\n", x);
```

```
}
```

```
struct Queue* enqueue(int x, int priority)
```

```
{
```

```
    struct Queue *t = (struct Queue*) malloc (sizeof(struct Queue));  
    t->data = x;  
    t->priority = priority;
```

```
    if (Front==NULL)
```

```
    {
```

```
        Front = t;  
        t->next = NULL;  
        Rear = t;  
        return Front;
```

```
    }
```

```
    struct Queue *p = Front;
```

```
    while(p->next != NULL && p->next->priority < t->priority )
```

```
    {
```

```
        p = p->next;
```

```
    }
```

```
    t->next = p->next;
```

```
    p->next = t;
```

```

    if (t->next == NULL)
    {
        Rear = t;
    }

    return Front;

}

struct Queue* dequeue()
{
    if (Front == NULL)
    {
        printf("Your stack is empty!\n");
        return NULL;
    }

    struct Queue *p = Front;
    Front = Front->next;
    free(p);

    return Front;
}

int peek()
{
    if (Front == NULL)
    {
        printf("Your stack is empty!\n");
        return -1;
    }
    return Front->data;
}

```

```

void display()
{
    struct Queue *po = Front;

    while (po != NULL)
    {
        printf("%d\t", po->data);
        po = po->next;
    }
    printf("\n");
}

```

Time:	Memory:
0.0000 secs	1.632 Mb

Your Output

```

10      102      110      100
10

```

```
#include <stdio.h>
```

```

struct Queue
{
    int data;
    struct Queue *next;
};

```

```

struct Queue *Front = NULL;
struct Queue *Rear = NULL;

```

```
struct Queue* enqueue(int x);  
struct Queue* dequeue();  
int peek();  
void display();
```

```
int main()  
{  
    Front = enqueue(10);  
    Front = enqueue(100);  
    Front = enqueue(110);  
    Front = enqueue(102);  
  
    //Front = dequeue();  
  
    display();  
  
    int x = peek();  
    printf("%d\n", x);  
  
    printf("Rear->next->data = %d\n", Rear->next->data);  
    printf("Front->data = %d\n", Front->data);  
  
}
```

```
struct Queue* enqueue(int x)  
{  
    struct Queue *t = (struct Queue*) malloc (sizeof(struct Queue));  
    t->data = x;  
  
    if (Front==NULL)  
    {  
        Front = t;  
        Rear = t;  
        Rear->next = Front;
```



```
    return Front;  
}
```

```
    Rear->next = t;  
    t->next = Front;  
    Rear = Rear->next;
```

```
    return Front;
```

```
}
```

```
struct Queue* dequeue()
```

```
{  
    if (Front == NULL)  
    {  
        printf("Your stack is empty!\n");  
        return NULL;  
    }  
}
```

```
    Rear->next = Front->next;  
    struct Queue *p = Front;  
    Front = Front->next;  
    free(p);
```

```
    return Front;
```

```
}
```

```
int peek()
```

```
{  
    if (Front == NULL)  
    {  
        printf("Your stack is empty!\n");  
        return -1;  
    }  
}
```

```

    return Front->data;
}

void display()
{
    struct Queue *po = Front;
    int flag = 0;

    while (po != Front || flag == 0)
    {
        flag = 1;
        printf("%d\t", po->data);
        po = po->next;
    }
    printf("\n");
}

```

Your Output

```

10      100      110      102
10
Rear->next->data = 10
Front->data = 10

```